COMPARISON · SPLIT · STRUCTURE

compare-code

Two fenced code blocks side-by-side, each with a label.

QUERY PATH · REPORT GENERATION

Before and after, the diff you can read aloud.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

Ten lines a side is the ceiling.

COMPARE-CODE · STRESS

BEFORE · AT THE PANE BUDGET

```
function beforePane(rows) {
  const out = [];
  for (const row of rows) {
    // ten lines per pane is the most
    // a side-by-side diff can hold
    // before the type drops below
    // back-row legibility
    out.push(transform(row));
  }
  return out;
}
```

AFTER · SAME BUDGET, SAME SHAPE

```
function afterPane(rows) {
  // the panes must stay line-comparable:
  // the eye pairs line N with line N
  // across the gutter — pad or trim
  // until the shapes align
  return rows.map(transform);
}
// past ten lines a side, split the
// comparison across two slides
```

QUERY PATH · REPORT GENERATION

Before and after, the diff you can read aloud.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

QUERY PATH · REPORT GENERATION

Before and after, the diff you can read aloud.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

QUERY PATH · REPORT GENERATION

Before and after, the diff you can read aloud.

BEFORE · ONE QUERY PER ROW

```
const signals = await db.signals.findAll();
for (const s of signals) {
  s.owner = await db.users.find(s.ownerId);
}
return signals;
```

AFTER · ONE BATCHED JOIN

```
const signals = await db.signals.findAll({
  include: { owner: true },
});
return signals;
```

When NOT to reach for compare-code.

One side is prose

If one column is code and the other is description, use a single fenced block with surrounding prose. compare-code is for code-versus-code.

Snippets longer than 14 lines

The text shrinks below readability past 14 lines per side. Split into two slides or extract the key delta into a smaller diff.

Three-way comparison

compare-code is binary. For three configurations or three implementations, use prose with successive fenced blocks or a `compare-table`.

RELATED COMPONENTS

See also.

`compare-prose` — the change is state, not code

`redline` — the comparison is prose-versus-prose

`redline` — the change is in verbatim text or legal language

`compare-table` — three or more variants on shared dimensions